

Ex no 4: Implementation of converting an Infix Expression to Postfix Expression

Aim:

Design, Develop and Implement a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, %(Remainder), ^(Power) and alphanumeric operands.

Algorithm:

- a. Initialize an empty stack for operators and an empty postfix string.
- b. Scan the infix expression from left to right:
- c. Operand → Append to postfix expression.
- d. Left Parenthesis (→ Push onto stack.
- e. Right Parenthesis) → Pop from stack to postfix until (is found, then discard (.
- f. Operator →
- g. While stack is not empty and operator at top has higher or equal precedence (consider associativity), pop from stack to postfix.
- h. Push current operator to stack.
- i. After scanning entire expression, pop all remaining operators from stack to postfix.
- j. Output the Postfix Expression
- k. End

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

// Push element to stack
void push(char c) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = c;
}
```

```

// Pop element from stack
char pop() {
    if (top == -1) {
        return '\0'; // empty stack
    }
    return stack[top--];
}

// Peek top element of stack

// Check precedence of operators

// Check associativity (Right = 1, Left = 0)
int isRightAssociative(char op) {
    if (op == '^') return 1;
    return 0;
}

// Function to convert infix to postfix
void infixToPostfix(char infix[], char postfix[]) {
    int i, k = 0;
    char c;

    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];

        // If operand, add to postfix
        if (isalnum(c)) {
            postfix[k++] = c;
        }
        // If '(', push to stack
        else if (c == '(') {
            push(c);
        }
        // If ')', pop until '(' is found
        else if (c == ')') {
            while (top != -1 && peek() != '(') {
                postfix[k++] = pop();
            }
            pop(); // remove '('
        }
        // If operator
        else {
            while (top != -1 && precedence(peek()) > 0 &&
                (precedence(peek()) > precedence(c) ||
                 (precedence(peek()) == precedence(c) && !isRightAssociative(c)))) {
                postfix[k++] = pop();
            }
            push(c);
        }
    }
}

// Pop remaining operators

```

```

while (top != -1) {
    postfix[k++] = pop();
}

postfix[k] = '\0';
}

// Main function
int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter an Infix Expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix Expression: %s\n", postfix);

    return 0;
}

```

OUTPUT

Enter an Infix Expression: A+B*C
 Postfix Expression: ABC*+

Result:

Thus, the program to convert an Infix Expression to a Postfix Expression was successfully designed, developed, and implemented in C.